09-configuring-rspec/04/configuring_rspec/expect_with_spec.rb
```
require 'wrong'

RSpec.configure do |config|
  config.expect_with :minitest, :rspec, Wrong
end

RSpec.describe 'Using different assertion/expectation libraries' do
  let(:result) { 2 + 2 }

  it 'works with minitest assertions' do
    assert_equal 4, result
  end

  it 'works with rspec expectations' do
    expect(result).to eq 4
  end

  it 'works with wrong' do
    # "Where 2 and 2 always makes 5..."
    assert { result == 5 }
  end
end
```

Here, we are using three different libraries: rspec-expectations, Minitest's assertions, and a great little library called Wrong.[12,13] (You will not normally need to use multiple assertion libraries; we are just demonstrating some different options.)

You can give expect_with a well-known library name—:rspec, :minitest, or :test_unit—or you can pass in a Ruby module containing the assertion methods you want to use. If you're writing your own module, your methods should signal a failure by raising an exception.

Other Useful Options

Before we wrap up this chapter, let's take a look at a few final options that don't fit into the categories we've described.

Zero Monkey-Patching Mode

The first option we'd like to highlight here is disable_monkey_patching!:

09-configuring-rspec/05/rspec_configure.rb
```
RSpec.configure do |config|
  config.disable_monkey_patching!
end
```

12. http://docs.seattlerb.org/minitest/
13. https://github.com/sconover/wrong